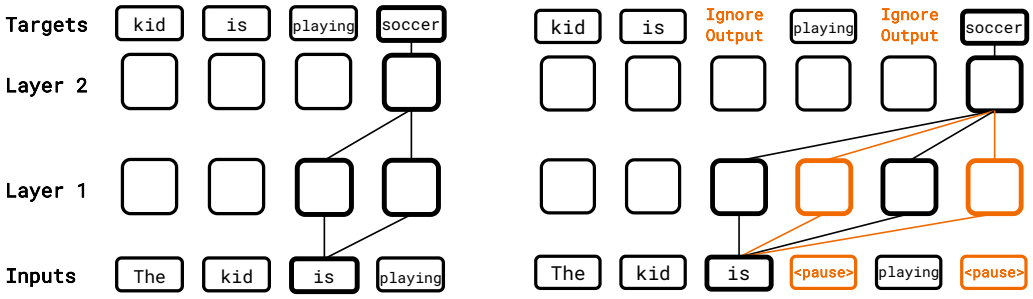

(a) Standard pretraining

(b) Pause-pretraining

Figure 2: **Standard vs. pause-pretraining**. We consider pretraining based on causal language modeling, where each token is predicted given all preceding tokens in the sequence, using unidirectional self-attention. Here, we visualize the computational pathways beginning from the token "`is`" on the input side of the decoder-only model, to a subsequent token "`soccer`" on the output side. Please see Figure 1 for a guide on how to follow this visualization. (a) In standard pretraining, we compute the model's loss at each output token, and backpropagate through it. (b) In pause-pretraining, we insert multiple copies of `<pause>` tokens at uniformly random locations in the input. However, we do not apply a loss on the model to predict these tokens, as indicated by each corresponding **`Ignore Output`** flags. This introduces new computational pathways connecting the input token and the output token of interest.

role the above characters play in natural language, we choose a *single* `<pause>` token residing outside of the standard vocabulary. To impose multi-token delays, we simply repeat this token. Building on this core idea, below we discuss our specific techniques for pause-pretraining and pause-finetuning.

Pretraining with the `<pause>` token The sequences in our pretraining data do not come with an annotation of which suffix constitutes the answer, since every input token also functions as a target output. Thus, it is impossible to execute the simple delaying strategy of appending dummy tokens before extracting the answer. Therefore, for a given pretraining sequence $\mathbf{p}_{1:N}$, we insert multiple `<pause>` tokens (say M_{pt} many) at uniformly *random* locations to obtain a pause-injected sequence, $\tilde{\mathbf{p}}_{1:N+M_{\text{pt}}}$. We visualize this in Figure 2b. We then train the model with the standard next-token prediction loss on this pause-injected sequence, while ignoring any loss term that corresponds to predicting the pause tokens themselves. Formally, let $S_{\text{ignore}} = \{k \colon \tilde{p}_{k+1} = \texttt{<pause>}\}$ denote the positions where the *next* token is a `<pause>` token. Then, for the decoder-only language model f, the pause-training loss is given by:

$$\mathcal{L}_{\text{PausePT}}(f, \tilde{\mathbf{p}}_{1:N+M_{\text{pt}}}) = \sum_{\substack{k=1 \\ k \notin S_{\text{ignore}}}}^{N+M_{\text{pt}}-1} \mathcal{L}_{\text{CE}}(\tilde{p}_{k+1}, f(\tilde{\mathbf{p}}_{1:k})), \tag{3}$$

where $\mathcal{L}_{\text{CE}}$ denotes the cross-entropy loss. Observe that the loss is skipped over indices in S_{ignore}. The rationale is that, we only want to use the `<pause>` tokens as a way of enforcing a delay in the model's computation; demanding that the model itself produce these tokens, would only be a pointless distraction. Finally, as is standard, we update the parameters of both the model and of all the tokens, including those of the `<pause>` token. We term this *pause-pretraining* (Algorithm 1).

Finetuning with the `<pause>` token In downstream finetuning, we are given a prefix $\mathbf{p}_{1:N}$ annotated with a target $\mathbf{t}_{1:T}$. Here, we append M_{ft} copies of the `<pause>` token to $\mathbf{p}_{1:N}$, to create our new prefix, $\tilde{\mathbf{p}}_{1:N+M_{\text{ft}}}$. We visualize how this introduces new computational pathways in Figure 1. As before, we ignore the model's outputs until the last `<pause>` token is seen. We apply the standard next-token prediction loss on the target with the new prefix, thus minimizing $\sum_{k=0}^{T-1} \mathcal{L}_{\text{CE}}(t_{k+1}, f([\mathbf{p}_{1:N+M_{\text{ft}}}, \mathbf{t}_{1:k}]))$, where $[\cdot]$ denotes the concatenation operation. Note that for any given downstream task, we fix M_{ft} to be the same across all inputs for that task. We again

update both the parameters of the model, and that of the whole vocabulary, including the <pause> token, as is standard. We term this *pause-finetuning* (Algorithm 2).

Pausing during inference During inference on the downstream task, we append M_{inf} <pause> tokens to the prefix and as always, we ignore the output of the model until the last <pause> token (Figure 1). We term this *pause-inference* (Algorithm 3).

3.2 Variants of Pause-Training

While pause tokens can be incorporated in either pretraining or finetuning, in our study, we will consider all combinations of this. Our hope is to identify if there are any differences in how each stage of pause-training affects inference-time performance. In total, we study four techniques:

1. Standard Pretraining and Standard Finetuning (**StdPT_StdFT**).
2. Standard Pretraining and Pause-Finetuning (**StdPT_PauseFT**): We train with <pause> tokens only during downstream finetuning. *If* this technique helps, it would promise a practically viable approach for pause-training off-the-shelf models.
3. Pause-Pretraining and Standard Finetuning (**PausePT_StdFT**): Here we introduce <pause> tokens during pretraining, but abandon it downstream. This is purely for analytical purposes (See §4.3).
4. Pause-Pretraining and Pause-Finetuning (**PausePT_PauseFT**): We inject delays into both stages.

Unless stated otherwise, we use the same number of pauses at inference as finetuning ($M_{\text{inf}} = M_{\text{ft}}$).

4 Experiments

Our main experiments broadly aim to address two questions:

(1) Does delaying the model computation via pausing help (hopefully, due to the wider computational flow), have no effect (since the tokens provide no new information, and substantially no new parameters are added) or hurt (perhaps, by distracting the model with stray tokens)?

(2) If at all these delays have any effect, is there a difference in performance when we inject it into the pretraining stage versus finetuning stage versus both?

4.1 Experiment Setup

We consider decoder-only models of size 1B and 130M for our main experiments. For our ablations, we stick to the 1B model. Both the standard and pause models are pretrained on the C4 English mixture (Raffel et al., 2020), using the causal next token prediction objective for a total of 200B tokens (slightly more than 1 epoch on C4). For pause-pretraining, we insert the <pause> token randomly at 10% of the sequence length (2048) positions, and trim the now-longer sequence to its original sequence length. We then conduct pause-pretraining and standard-pretraining for the same number of total tokens (200B). We use a *single* <pause> token embedding, effectively increasing the parameter count by 1024 (the token embedding size), a quantity that is dwarfed by the 1 billion total parameter count (the token constitutes a 10^{-6} fraction of the model size).

Since we expect different downstream tasks to benefit from a different number of finetuning <pause> tokens M_{ft}, we run finetuning with M_{ft} (and likewise M_{inf}) set to 10 and 50 and report the best of these two for our consolidated results. However, we provide the values for both $M_{\text{ft}} \in \{10, 50\}$ in Appendix E, in addition to a more finegrained ablation of this hyperparameter in Section 5. For all the downstream finetuning experiments, we report mean and standard deviation over 5 runs (with the randomness purely from the finetuning stage). We tune the learning rate and batch size for standard end-to-end training, and use the best hyperparameter for all other training variants as well. We share all the hyperparameters in Appendix H.

4.2 Downstream datasets

We consider nine varied downstream tasks: (a) reasoning (GSM8k (Cobbe et al., 2021)), (b) extractive question answering (SQuAD (Rajpurkar et al., 2016), CoQA (Reddy et al., 2019)), (c) general understanding (CommonSenseQA (Talmor et al., 2019), PhysicalIQA (Bisk et al., 2020)), (d)

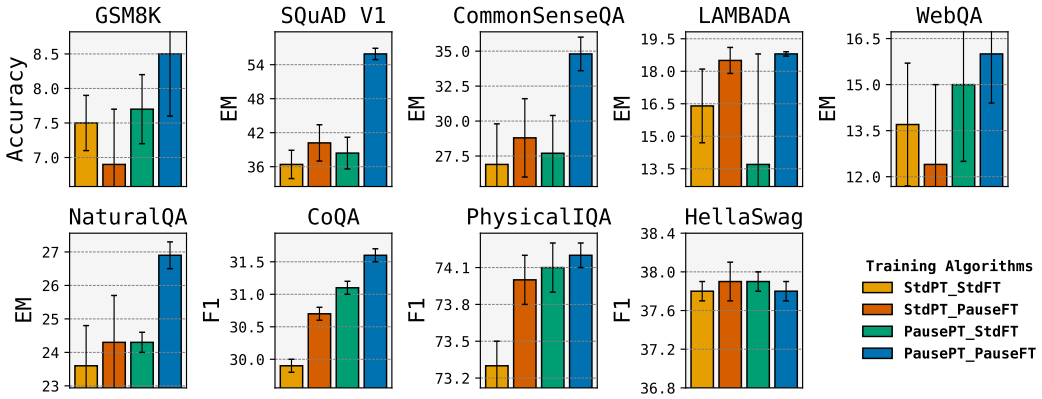

Figure 3: **Downstream performance for a 1B model.** Injecting delays in both stages of training (PausePT_PauseFT) outperforms the standard end-end training StdPT_StdFT on our wide variety of tasks (except HellaSwag). In contrast, introducing delays only in the finetuning stage provides only lukewarm gains, and even hurts in GSM8k.

long term context recall (LAMBADA (Paperno et al., 2016)), (e) natural language inference (HellaSwag (Zellers et al., 2019)), and (f) fact recall (WebQuestions (Berant et al., 2013), Natural Questions (Kwiatkowski et al., 2019)). HellaSwag and PhysicalIQA are scoring tasks. We note that our implementation of CommonSenseQA is as a decoding task, and hence we report Exact Match (EM) scores. Detailed dataset description is in Appendix G.

4.3 Results: Effect of pause-training

We report the performance of the four approaches in §3.2 on all our downstream tasks for our 1B model in Figure 3, and for our 130M model in Appendix B. We discuss zero-shot results in §D.

The benefit of pause-pretraining followed by pause-finetuning (PausePT_PauseFT). Our first core finding is that there are clear gains when <pause> tokens are introduced during both pretraining and finetuning (PausePT_PauseFT), across a majority of the tasks we consider. Overall, this outperforms the standard baseline (StdPT_StdFT) on eight tasks for the 1B model, and on six tasks for the 130M model (Appendix Fig 5) albeit to varying extents. Most prominently, for the 1B model on the SQuAD question-answering task, PausePT_PauseFT improves over StdPT_StdFT by an 18% EM score. Similarly, we observe upto 8% gains on the general understanding task of CommonSenseQA. On the reasoning task of GSM8k, PausePT_PauseFT gives an accuracy of 8.5% compared to 7.5% of the standard baseline. Similar gains are observed in other tasks like long-term context understanding (LAMBADA) and also on fact recall tasks like WebQA and NaturalQuestion.

The lukewarm effect of pause-finetuning a standard-pretrained model (StdPT_PauseFT). In contrast to the above observations, introducing delays only during downstream finetuning (StdPT_PauseFT) gives mixed results. While there are gains on about 5 benchmarks, they are comparitively less. On the remaining, the performance mirrors (or is worse than) standard training.

Isolating the benefits of pause-pretraining independent of downstream delay (PausePT_StdFT). The gains in PausePT_PauseFT may come not only from inference-time delays, but also from better representations learned by pause-pretraining. To isolate the latter effect, we consider PausePT_StdFT, where we do not inject delays in the downstream task. Here the gains are clear only in two tasks (CoQA and PhysicalIQA). Thus, we conclude that pause-pretraining improves the representation for a few downstream tasks; conversely, in most tasks, the gains of PausePT_PauseFT must come from well-learned delayed computations executed at inference-time.

Filler characters as <pause>: For completeness, we also report results for inference on StdPT_StdFT models, delayed with 10 or 50 periods ('.'). Corroborating the observations of Lanham et al. (2023), we find no gains in doing this (Figure 4a).